# Table of Contents

Steps specifications . . . . . 1
- Configuring a step specification . . . . . 1
- Fan-out . . . . . 2
- Fan-in / Joins . . . . . 3
- Error processing steps . . . . . 4
- Core operators . . . . . 5

# Steps specifications

## Configuring a step specification

Depending on their nature, you can define steps specifications as root of a scenario or successor of another step specification.

```
scenario("hello-netty-udp-world") {
        minionsCount = 100
        rampUp {
            // Starts all at once.
            regular(100, minionsCount)
        }
    }
    .start()
    .netty()

    .tcp { ①
        name = "my-tcp"
        connect {
            address("localhost", availableTcpPort)
            noDelay = true
        }
        request { tcpRequest1 }
        metrics { connectTime = true }
        events { connection = true }
    }

    .reuseTcp("my-tcp") { ②
        name = "reuse-tcp"
        iterate(20, Duration.ofMillis(100))
        request { tcpRequest2 }
    }
```

① The `tcp` step is defined as first step of the graph, differently said as root - hence, it expects `Unit` as input

② The `reuseTcp` step is defined as successor of `tcp` and receives its output as input.

There are two ways of configuring a step specification. Some receive the configuration closure as parameter like the `tcp step above`. Generally, you can also call `configure` after a step to configure it:

```
reuseTcp("my-tcp") {
    request { tcpRequest2 }
}.configure {
    name = "reuse-tcp" ①
    iterate(20, Duration.ofMillis(100)) ②
    retryPolicy(instanceOfRetryPolicy) ③
    timeout(2000) ④
}
```

① Set a name to your step: this is convenient for analysis of events, successes, failures - the same name can be used in several scenario specifications, which helps for cross-analysis and reusability.

② Repeat the step 20 times with an interval of 100 ms between one execution and the next one.

③ Step-own retry policy, see [Retry policy] for more details.

④ Timeout to complete the step execution, in ms. Note that if a retry policy is set that allows another attempt, a timeout error will lead to a new execution.

# Fan-out

When you want to split a branch and add two concurrent steps as successors, you can use the `split`:

```
execute { }
    .split { ①
        map { ... } ②
            .validate{ ... }

        flatten() ③
            .filterNotNull()

    }
```

① `split` takes a closure with the step `anyStep` as receiver.

② The first branch receives the output of `anyStep` as input.

③ The second branch also receives the output of `anyStep` as input but does something different with.

Here is the graphical representation:

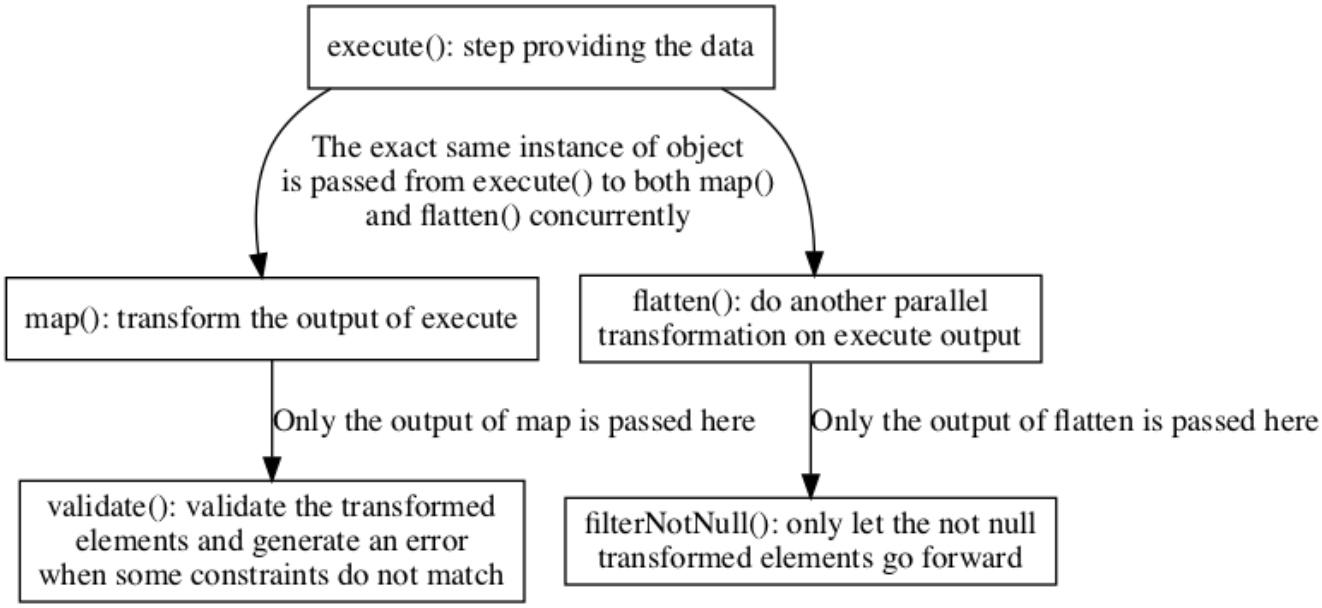



*Figure 1. Fan-out*

Both branches run concurrently, their no defined execution order.

If the entity returned by `anyStep` is mutable, take care of not altering it in one branch, because this would have side-effects on the other. Prefer creating deep clones.

# Fan-in / Joins

Because you split a scenario into different branches or want to verify data from different sources, you often need to join them. You can either do it using the name of a concurrent step:

```
execute { } ①
    .innerJoin<EntityWithKey, EntityWithId>(
        on = { "${it.key}" }, ②
        with = "my-datasource-poll-step", ③
        having = { it.id } ④
    )
    .map { } ⑤
```

① `execute` provides an output of type `EntityWithKey`, with a field `key` being an Int. We call this step the "left" step.

② We provide a calculation of the key from the left side and decide to convert it as a String.

③ The step called `my-datasource-poll-step` is declared in a parallel branch (or earlier in the same branch) and provides an output of type `EntityWithId` with a field `id` being a String.

④ Closure to extract the key from the right side, being already a String.

⑤ The result is a `Pair<EntityWithKey, EntityWithId>` where string representation of `key` of the `EntityWithKey` equals to the field `id` of the `EntityWithId`.

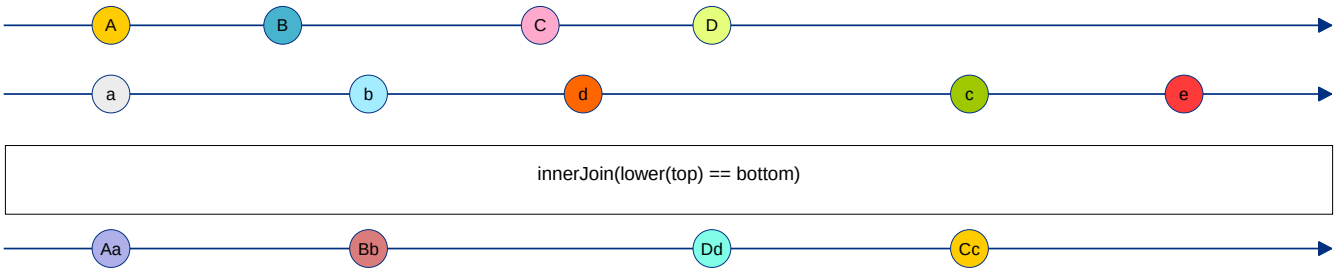
Records coming from the right side are kept in a cache until one comes from the left side with the

same key. Once used, they are evicted from the cache.

When a flow comes from the left side but has no corresponding record yet coming from the right, it waits until the defined timeout on the step. See Configuring a step specification for more details.

The step context after the join inherits from the one coming from the left.

Here is the graphical representation:




You can also create a step directly in the join:

```
execute { }
    .innerJoin<EntityWithKey, EntityWithId>(
        on = { "${it.key}" },
        with = { ①
            r2dbc().poll{
                // ...
            }
        },
        having = { it.id }
    )
    .map {}
```

① Things are working exactly the same as in the first example, but the step is directly created here. The receiver of the `with` is the scenario.

# Error processing steps

When an error occurs when executing a step, after all the potential retries, the step context being carried from top to bottom is marked as "exhausted".

From that point of the process, only the steps relevant for error processing are executed, the others are simply bypassed.

Those steps from the core are:

- `catchError`: takes the collection of errors as parameter to process them.
- `catchExhaustedContext`: takes the full exhausted step context as parameter, where you can also set the `isExhausted` flag to `false` in order execute next steps.

# Core operators

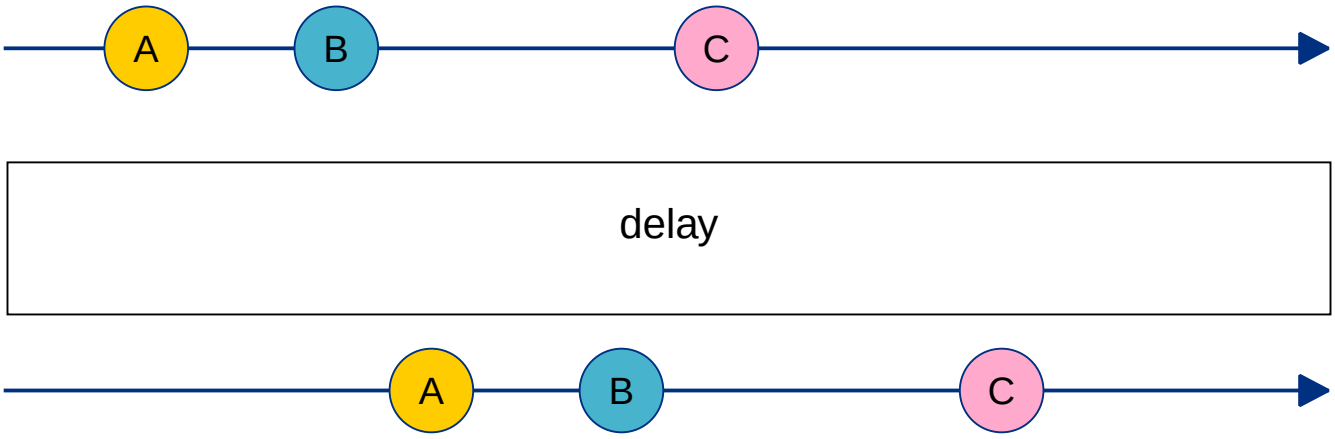
QALIPSIS provides default operators to tweak the execution of your scenario, that do not require the addition of plugins to your classpath.

While you already know some of them described just above, others are available to let you transform, filter, verify the data, alter the behavior...

## Timing operators

### delay

Adds a constant delay before executing the next step, keeping the same execution pace as previous step.

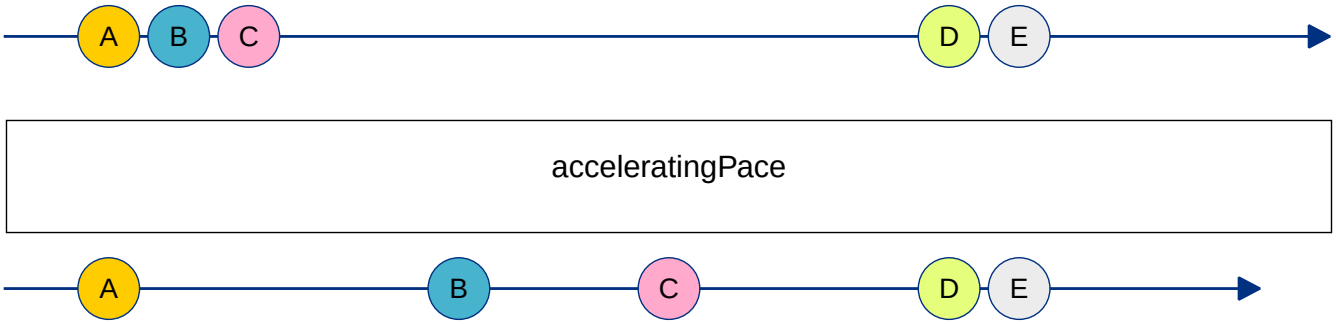



### pace

Ensures that the next step is executed at an expected pace, whatever the one of the input is. The calculation applies independently to each minion.

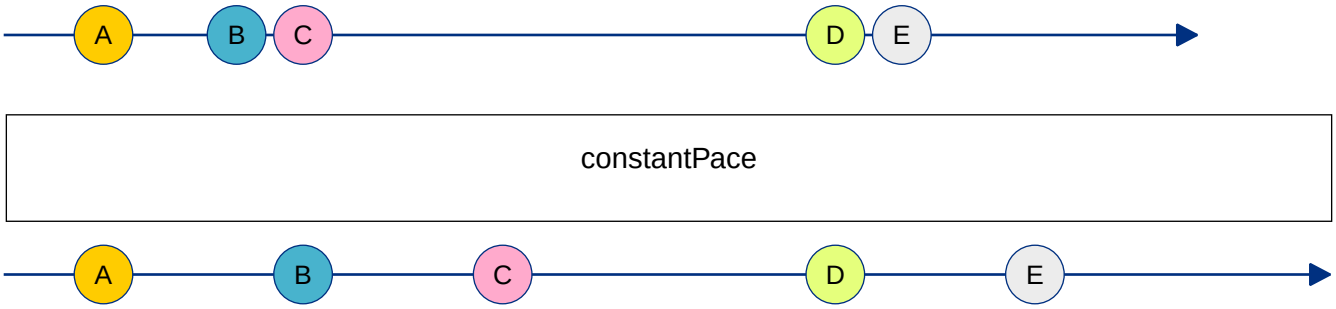
### acceleratingPace

Like pace, but reducing the waiting interval at each iteration. Too slow down the pace, you can simply use an accelerator lower than 1.



### constantPace

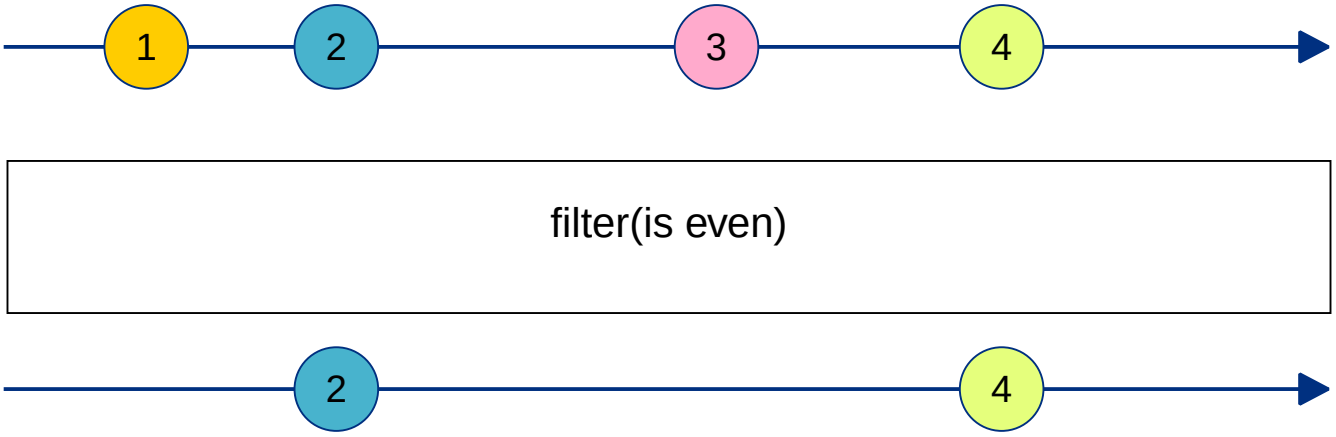
Applies a constant pace to execute the step after.




## Filter operators

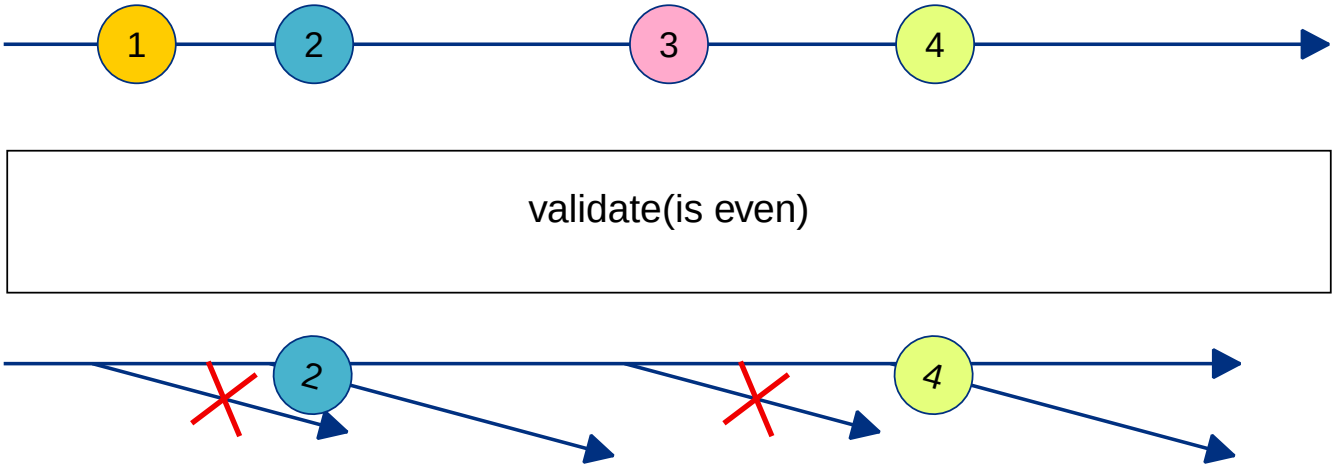
### filter

Removes the step contexts having an input not matching the filter specification. It can be seen as a silent equivalent of validate, because it does neither generate any error, nor mark the step context as exhausted.



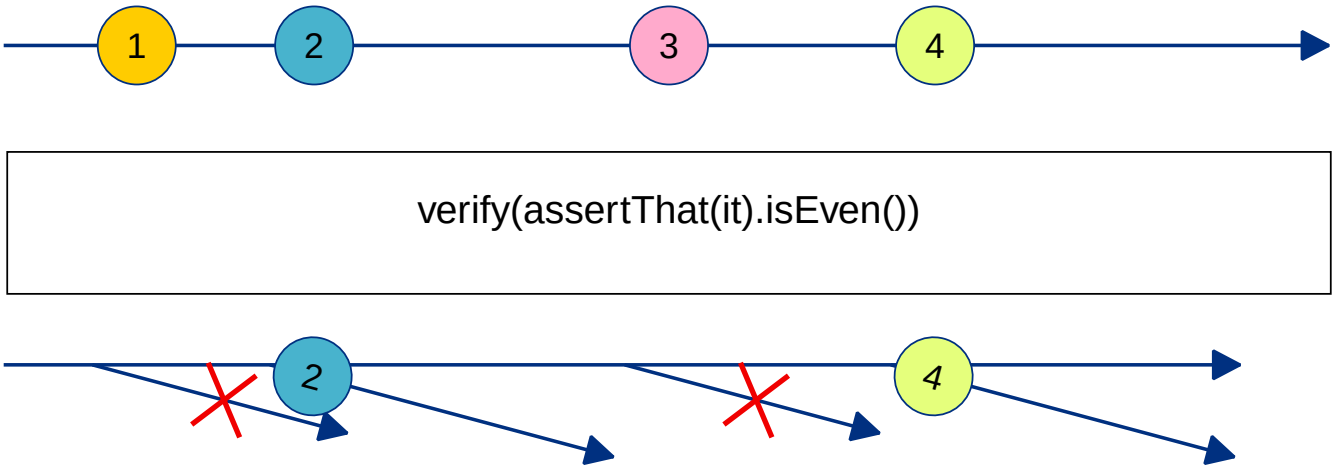

### validate

Validates the input. When errors are returned, the step context is marked as exausted and can then only be processed by error processing steps.




**verify**

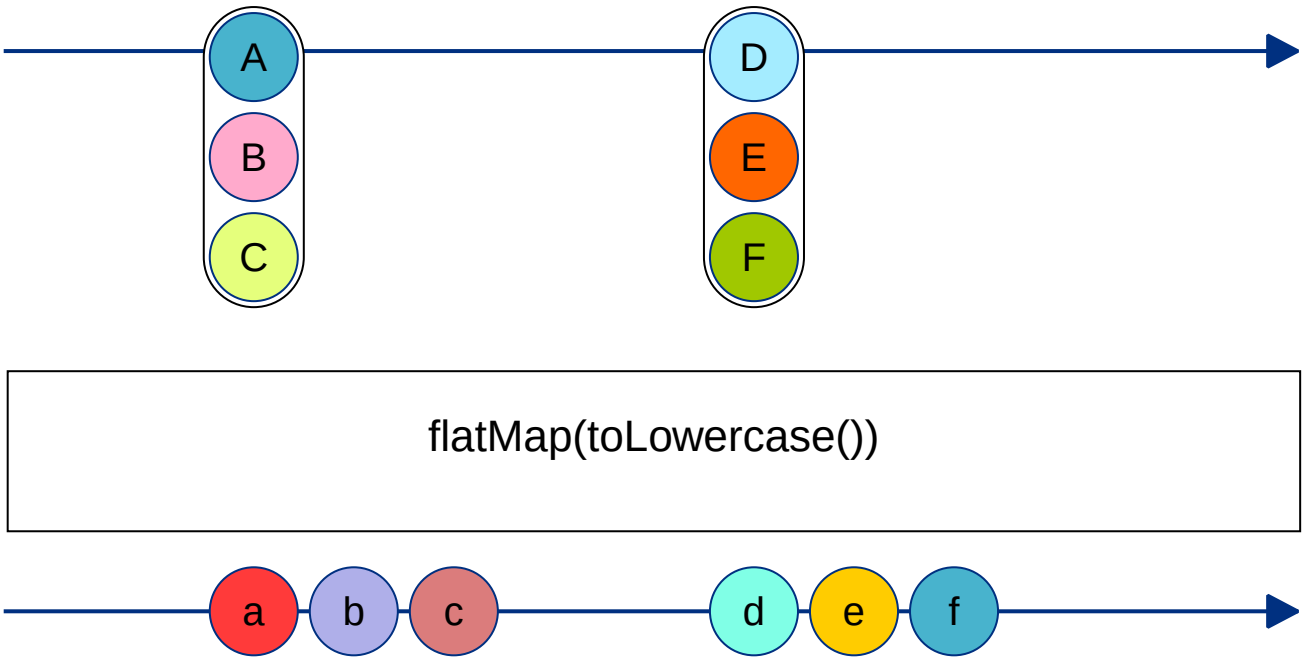
Exactly like validate, but meant to be used for assertions instead of data quality / safety.




## Transformation operators

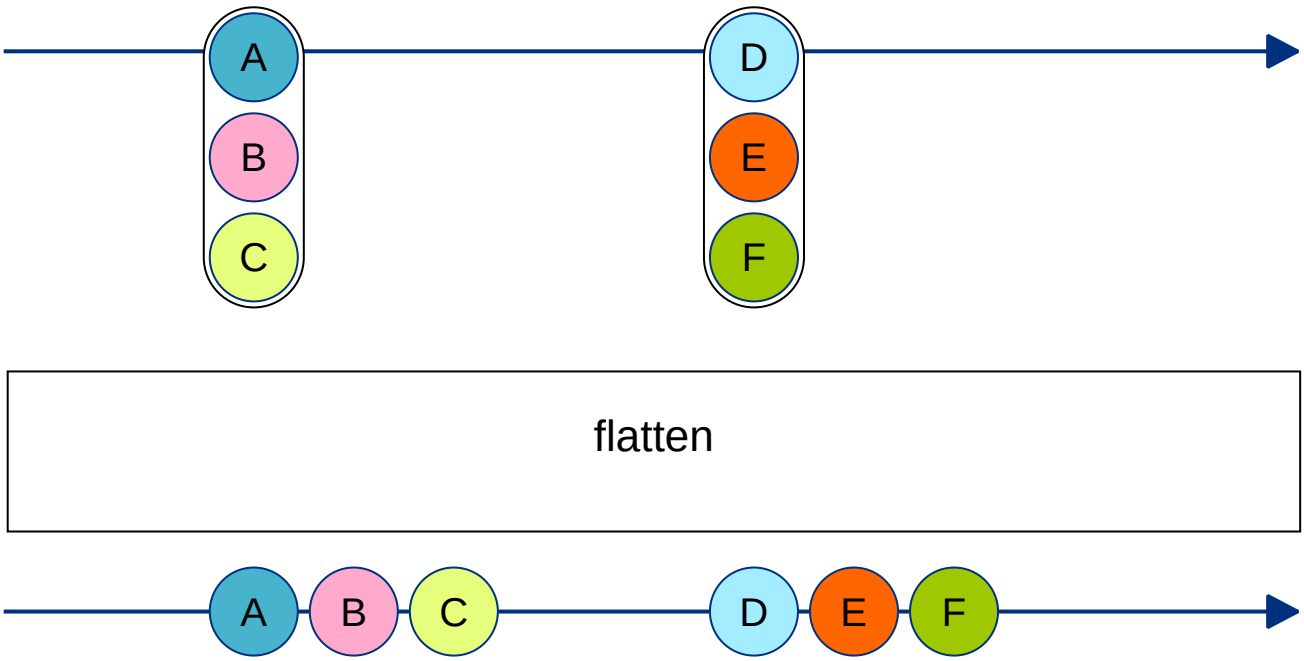
**flatMap**

Converts a collective input into single element in the output, using the user-defined strategy.



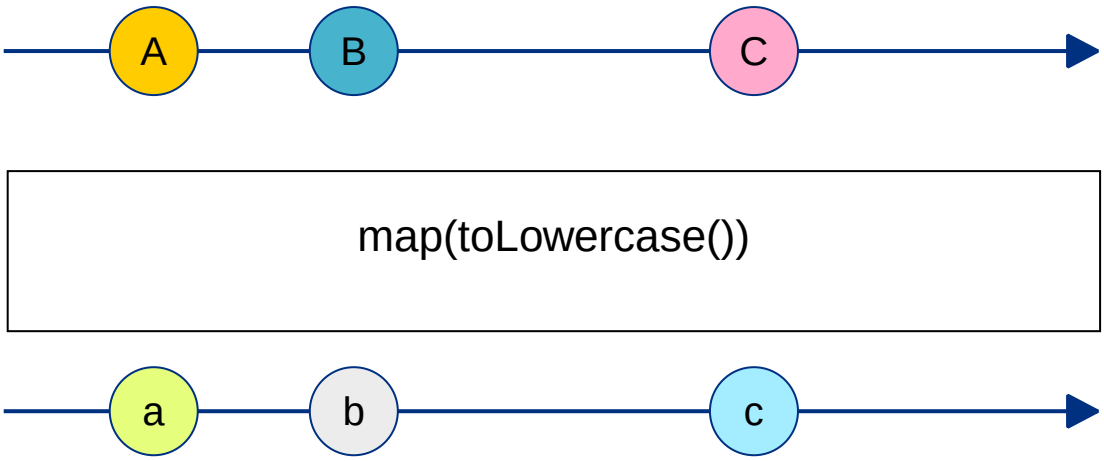

**flatten**

Converts a collective input into single element in the output, for a known collective type (array, iterable).



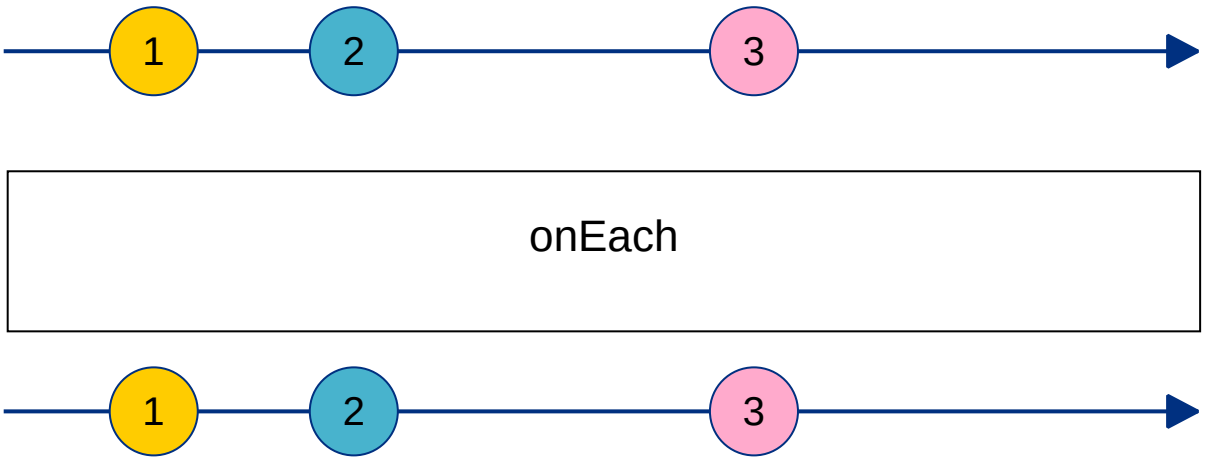

**map**

Converts each element into another in the output. `mapWithContext` provides the same capability with the step context as additional argument.




**onEach**

Executes one or several statements on each element of the input, and forwards them unchanged to the output.




## Caching operators

**shelve**

Caches the provided values in a shared cache.When using a distributed architecture, make sure the cached value can be serialized by the used implementation of shared state registry.The ID of the minion is automatically added to the key to avoid collusion.

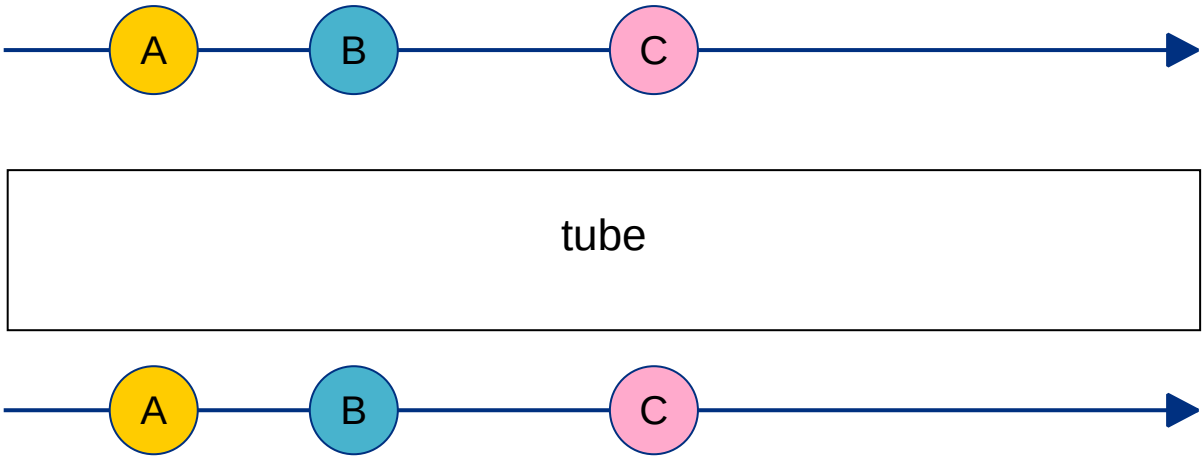
**unshelve**

Fetches one or more record from the shared state registry, previously shelved with the provided keys for the same minion.

## Utility operators

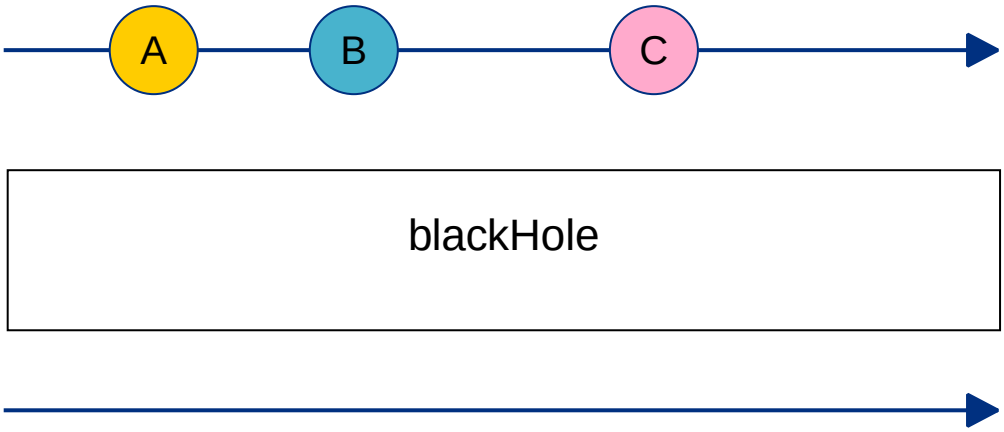
**tube**

Simply forwards the input to the output.




**blackHole**

Consumes the input and provides no output.




**stage**

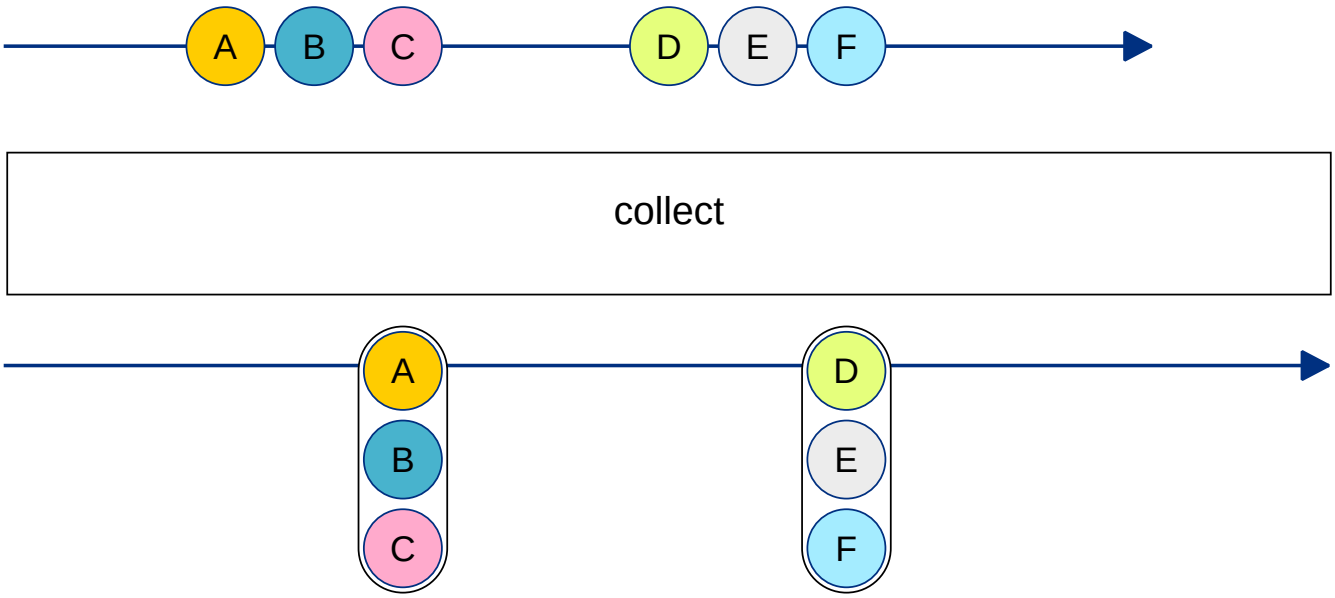
Groups a linear graph of steps. Stages are not mandatory in a scenario, but provide a convenient way to iterate, waiting for the last step to be completed before looping back.

```
stage("my-stage") {
    map { it * 3 }.map { arrayOf(it, it + 100) }.flatten() ①
}.configure { ②
    iterate(5)
}
```

① Create a chain of steps, where no `split()` is allowed.

② You can configure the whole stage to repeat from its beginning only once the tail step is completed

## Combining operators

### collect

Collects inputs across all the minions and releases them as a list, within the latest received step context. This operator can be seen as the opposite of the `flatten`.